

iAgenda

Auditoría de Código

by iAgentes

Fecha: 26 de junio de 2026

Proyecto: D:\agendamiento\apps\web

Stack: Next.js 16 · React 19 · TypeScript · Prisma · PostgreSQL (Neon) · Upstash Redis · Clerk · Tailwind v4 · Cal.com · OpenAI · WhatsApp API

Propósito: SaaS multi-tenant colombiano para gestión inteligente de reservas de restaurantes.

CONFIDENCIAL

Resumen Ejecutivo

Arquitectura general: ' Sólida
Seguridad: & p 4 hallazgos críticos
Type Safety: Ø=Ý4 2 hallazgos críticos
Testing: Ø=Ý4 0% de cobertura
Mantenibilidad: & p Media
Performance: Ø=Üì Mejorable

La auditoría encontró 6 hallazgos críticos, 5 de alta prioridad, 5 de prioridad media y 3 sugerencias. Se recomienda atender los críticos antes de cualquier despliegue a producción.

Prioridad Crítica (6 hallazgos)

#1: ignoreBuildErrors: true en next.config.ts

Archivo: next.config.ts:5 | Esfuerzo: Medio

La configuración de Next.js tiene typescript.ignoreBuildErrors: true, desactivando el type-checking en build.

Por qué importa:

- Cualquier error de tipo (undefined no manejado, null checks, tipos incorrectos) pasa desapercibido hasta runtime en producción.
- En un SaaS multi-tenant que maneja pagos, reservas y datos personales (Ley 1581), un error silencioso puede significar 500s para clientes pagos o reservas perdidas.
- Integraciones críticas (Stripe, Wompi, WhatsApp) dependen de tipos correctos para funcionar.

Plan de acción:

- Cambiar ignoreBuildErrors a false
- Ejecutar npx tsc --noEmit para inventariar errores
- Corregir errores: primero any types, luego null-checks, luego tipos incorrectos
- Integrar tsc --noEmit en CI/CD

#2: Salt fijo y fallback público en cifrado AES-256-GCM

Archivo: encryption.ts:10-12 | Esfuerzo: Bajo-Medio

PBKDF2 usa salt fijo hardcodedo y la clave por defecto es pública en el código fuente.

Por qué importa:

- Salt fijo produce la misma clave derivada siempre. Dos textos iguales producen ciphertexts idénticos, rompiendo AES-GCM.
- Clave pública: cualquiera con acceso al código puede descifrar WhatsAppAgent.accessToken, stripeSecretKey y wompiPrivateKey almacenados en BD.
- 10,000 iteraciones PBKDF2 está muy por debajo del estándar actual (OWASP recomienda 600,000+).

Plan de acción:

- Hacer ENCRYPTION_KEY obligatoria en producción con lanzamiento de error si falta
- Eliminar salt fijo: usar IV aleatorio de 12 bytes (AES-GCM ya lo provee)
- Aumentar iteraciones PBKDF2 a 600,000+ o migrar a crypto.scryptSync
- Rotar (re-cifrar) todos los datos cifrados actualmente con la clave vieja

#3: API Keys hasheadas con SHA-256 sin salt

Archivo: api-key.ts:16 | Esfuerzo: Medio

Las API keys se almacenan como SHA-256(rawKey) directo, sin salt aleatorio.

Por qué importa:

- SHA-256 sin salt es vulnerable a rainbow tables. Un atacante con acceso a la BD puede precomputar hashes.
- Las API keys otorgan acceso completo a: crear/ver/cancelar reservas y datos de clientes.
- No hay rate limiting en intentos de autenticación, permitiendo fuerza bruta offline.

Plan de acción:

- Migrar a crypto.scryptSync(rawKey, salt, 32) con salt aleatorio de 16 bytes
- Almacenar en formato salt:hash dentro de keyHash
- Implementar backward compatibility para keys existentes (detectar formato antiguo y migrar en caliente)
- Agregar rate limiting (5 intentos/segundo/IP) en authenticateApiKey

#4: Exposición de error.message en respuestas HTTP

Archivo: api/bookings/route.ts:220 | Esfuerzo: Bajo

Los catch handlers devuelven `error.message` directamente al cliente HTTP en múltiples endpoints.

Por qué importa:

- Filtra información interna: stack traces, rutas de servidor, configuraciones, versiones de librerías.
- En el contexto colombiano, Ley 1581 y CONPES 3995 recomiendan no exponer información técnica en errores.
- Se repite en webhooks, API v1 y dashboard. Es un vector de reconocimiento para atacantes.

Plan de acción:

- Crear helper `createErrorResponse`: en producción devuelve mensaje genérico, en desarrollo el original
 - Reemplazar todos los catch blocks que exponen `error.message` en el proyecto
 - Loggear errores reales con `console.error` estructurado o Sentry
-

#5: Uso generalizado de tipo `any`

Archivo: Múltiples archivos | Esfuerzo: Alto

Uso extensivo de `any` en booking route, WhatsApp webhook, dashboard layout, audit y email.

Por qué importa:

- Cada `any` es una desactivación manual del sistema de tipos. El compilador no puede detectar errores.
- `body: any` en booking route acepta payloads malformados sin advertencia.
- `Record<string, any>` en dashboard/layout.tsx para iconos: nombre mal escrito rompe UI sin error.
- `restaurant: any` en WhatsApp webhook obliga a accesos encadenados frágiles sin protección.

Plan de acción:

- Tipar `body` con `Zod safeParse` (los esquemas ya existen en `api-schemas.ts`)
 - Reemplazar `Record<string, any>` con `Record<string, Lucidelcon>`
 - Tipar `restaurant`, `messages`, `args` con tipos generados por Prisma
 - Agregar regla ESLint `@typescript-eslint/no-explicit-any`: error
-

#6: Sin tests automatizados

Archivo: Todo el proyecto | Esfuerzo: Alto

No existe ningún test (unitario, integración, e2e) en todo el proyecto.

Por qué importa:

- No hay red de seguridad para refactors. Funciones complejas como `parseLocalDateInTimezone`, `authenticateApiKey` y `processWithOpenAI` no tienen verificación.
- El flujo `POST /api/bookings` es una saga distribuida con 4 sistemas (Redis, Cal.com, PostgreSQL, Stripe) que necesita tests de integración.
- Sin tests no se puede implementar CI/CD con confianza. Cada deploy es riesgoso.

Plan de acción:

- Configurar Vitest con mocks de Prisma, Redis y fetch (msw)
 - Escribir tests unitarios: `encrypt/decrypt`, `hashApiKey`, `signPayload`, `sanitizePayloadForLog`
 - Escribir tests de integración: `POST /api/bookings`, `dispatchWebhookEvent` + worker
 - Integrar en CI: `npm run test` en cada PR con coverage mínimo 80%
-

Prioridad Alta (5 hallazgos)

#7: parseLocalDateInTimezone frágil para DST

Archivo: api/bookings/route.ts:258-293 | Esfuerzo: Bajo

Función de 36 líneas que calcula offsets de zona horaria manualmente, incorrecta para zonas con DST.

Por qué importa:

- El approach asume que el offset UTC es bidireccionalmente correcto, pero falla en zonas con Daylight Saving Time.
- Un restaurante en Santiago de Chile (DST) podría tener bookings con ± 1 hora de error.
- date-fns-tz (fromZonedTime) ya podría usarse y resuelve esto con 2 líneas en vez de 36.

Plan de acción:

- Reemplazar con fromZonedTime de date-fns-tz
 - Agregar tests con casos borde: DST, medianoche, UTC+5:30
 - Buscar patrones similares de parsing manual en el proyecto
-

#8: Zero validación Zod en endpoint POST /api/bookings

Archivo: api/bookings/route.ts vs api-schemas.ts | Esfuerzo: Bajo

Existen esquemas Zod en api-schemas.ts pero el endpoint crítico no los usa.

Por qué importa:

- Validación manual débil: solo verifica existencia, no formato (email, fecha, partySize).
- Código duplicado: los esquemas Zod ya están escritos y completos pero no se sincronizan.
- Sin safeParse no hay errores descriptivos para el cliente API.

Plan de acción:

- Usar CreateBookingSchema.safeParse(body) en POST /api/bookings
 - Hacer de api-schemas.ts la fuente única de verdad compartida
 - Agregar tests que verifiquen que el schema rechaza payloads inválidos
-

#9: Redis Proxy miente sobre locks NX

Archivo: redis.ts:34-36 | Esfuerzo: Medio

El Proxy de Redis devuelve OK en locks NX aunque Redis esté caído.

Por qué importa:

- Si Redis falla, el proxy dice lock adquirido cuando no lo está. Dos reservas simultáneas pueden duplicarse.
- Impacto directo en el negocio: cliente llega y su mesa está ocupada. Mala experiencia y posible compensación.
- El patrón de Proxy oculta fallos de infraestructura que deberían ser visibles.

Plan de acción:

- Eliminar el Proxy o implementar fallback consciente con SELECT ... FOR UPDATE NOWAIT en PostgreSQL
 - Agregar monitoreo de salud de Redis (health check, métricas)
-

#10: Rate limiting por restaurantId en vez de por keyId

Archivo: api-auth.ts:135-141 | Esfuerzo: Bajo

Todas las API keys de un restaurante comparten el mismo contador de rate limit.

Por qué importa:

- Si un restaurante tiene 3 keys (web, POS, integrador), el límite se agota 3x más rápido.
- Una key maliciosa puede hacer rate-limit a las keys legítimas del mismo restaurante.
- No hay aislamiento entre environments live y test.

Plan de acción:

- Cambiar clave de rate limit a key:\${apiKey.id} en vez de restaurant:\${restaurantId}
 - Agregar límite agregado secundario por restaurante como soft limit con alerta
-

#11: WhatsApp webhook monolítico (428 líneas)

Archivo: api/webhooks/whatsapp/route.ts | Esfuerzo: Medio

Un solo archivo con 5 responsabilidades: HMAC, rate limiting, DB, agente OpenAI, Meta API.

Por qué importa:

- Cambiar el prompt del agente requiere modificar el mismo archivo que la verificación HMAC.
- Para testear hay que mockear 5 sistemas simultáneamente: es virtualmente imposible.
- Args sin tipar (any) + JSON.parse de OpenAI puede producir errores en runtime.

Plan de acción:

- Extraer buildSystemPrompt -> src/lib/whatsapp/prompts.ts
 - Extraer processWithOpenAI -> src/lib/whatsapp/agent.ts
 - Extraer tools -> src/lib/whatsapp/tools.ts
 - Extraer sendWhatsAppMessage -> src/lib/whatsapp/messenger.ts
 - Dejar en route.ts solo: orquestación de alto nivel
-

Prioridad Media (4 hallazgos)

#12: HTML templates inline como strings multilínea

Archivo: email.ts:28-46, 86-143 | Esfuerzo: Medio

Templates de correo en strings HTML multilínea con interpolación directa sin escape.

Por qué importa:

- XSS: restaurantName con `<script>` se inyecta en correos que reciben usuarios legítimos.
- Sin syntax highlighting ni validación de cierre de etiquetas. Div abierto rompe layout sin aviso.

Plan de acción:

- Escapar variables con función `escapeHtml()` como solución rápida
- Migrar a react-email (compatible con Resend, JSX real) como solución definitiva

#13: Promise.all sin límite de concurrencia

Archivo: webhook-queue.ts:86-103 | Esfuerzo: Bajo

`dispatchWebhookEvent` lanza todos los webhooks simultáneamente sin límite.

Por qué importa:

- 50 webhooks en paralelo = 50 conexiones concurrentes. Si el receptor está lento, satura recursos.
- Si todos fallan, BullMQ reintenta 5 veces en paralelo con backoff exponencial.

Plan de acción:

- Usar `p-limit` para limitar a 5 envíos concurrentes
- Configurar `concurrency` en el worker de BullMQ

#14: Sin paginación en API routes del dashboard

Archivo: API routes del dashboard | Esfuerzo: Medio

Los endpoints de listado no implementan paginación (`findMany` sin `take/skip/cursor`).

Por qué importa:

- Con 10,000+ clientes, la query se vuelve progresivamente más lenta.
- Serverless functions de Vercel tienen timeout máximo. Query sin límite puede excederlo.
- El usuario espera todos los datos antes de ver algo, o renderiza una lista enorme.

Plan de acción:

- Agregar `take` (default 50) y `cursor-based pagination` en endpoints GET
- Devolver metadata: `{ data, nextCursor, total }`
- Implementar scroll infinito con React Query + `keepPreviousData`

#15: Import dinámico innecesario de OpenAI

Archivo: whatsapp/route.ts:288 | Esfuerzo: Bajo

OpenAI se importa dinámicamente dentro de la función `processWithOpenAI`.

Por qué importa:

- OpenAI está en `dependencies`, no en `devDependencies`. El bundle ya lo incluye.
- El import dinámico agrega ~50-100ms de latencia a cada mensaje WhatsApp.
- En fines de semana con alta rotación, se acumulan segundos de latencia.

Plan de acción:

- Cambiar a import estático al inicio del archivo
- Mover inicialización del cliente a singleton fuera de la función

Sugerencias (3 hallazgos)

#16: ICON_MAP sin tipo correcto

Archivo: dashboard/layout.tsx:20 | Esfuerzo: Bajo

Record<string, any> en vez de Record<string, LucideIcon>.

Por qué importa:

- Pierde type safety: un nombre mal escrito en NAV_ITEMS rompe la UI sin error de compilación.

Plan de acción:

- Cambiar el tipo a Record<string, LucideIcon>.
-

#17: Comentarios mixtos español/inglés

Archivo: Varios | Esfuerzo: Bajo

Comentarios en ambos idiomas, a veces en el mismo archivo.

Por qué importa:

- Los comentarios en inglés tienden a desactualizarse más rápido en un equipo hispanohablante.

Plan de acción:

- Estandarizar progresivamente a español, priorizando archivos modificados frecuentemente.
-

#18: Mocks incluidos en src/

Archivo: src/mocks/* | Esfuerzo: Bajo

Archivos de mock dentro de src/ se incluyen en build de producción.

Por qué importa:

- Pueden empaquetarse accidentalmente en el build de producción, aumentando el bundle size y potencialmente exponiendo datos de prueba.

Plan de acción:

- Mover a __tests__/mocks/ o excluir del tsconfig.
-

' Buenas Prácticas Encontradas

- Singleton de Prisma con globalThis — evita múltiples conexiones en desarrollo
 - Rate limiting multi-capa en endpoints críticos y API Keys
 - HMAC-SHA256 para firma de webhooks salientes
 - Sanitización PII (Ley 1581 Colombia) en webhook deliveries
 - Idempotencia en webhooks vía idempotencyKey + upsert en BD
 - Suspensión automática + notificación por email de webhooks caídos
 - SDK público tipado con retry automático y rate limit handling
 - AES-256-GCM para cifrado de datos sensibles (algoritmo correcto)
 - Zod + OpenAPI para documentación de API pública
 - Fire-and-forget para logging de auditoría sin bloquear flujo principal
 - Optimistic locking con Redis NX para prevenir double-booking
 - Clerk middleware para protección de rutas del dashboard
 - Planes FREE/PRO/ENTERPRISE con límites de rate diferentes
-

Cronograma Sugerido

Semana 1 — Ø=Ý% Seguridad
Items 1, 2, 3, 4

Semana 2 — Ø=Ý% Type Safety + Tests
Items 5, 6 (setup + unitarios)

Semana 3 — & p Correctitud
Items 7, 8, 9

Semana 4 — & p Arquitectura
Items 10, 11

Semana 5 — Ø=Üì Mejoras
Items 12, 13, 14, 15

Semana 6 — Ø=Üì + Ø=Üj Final
Items 16-18 + tests integración

Total estimado: 6 semanas. Cada semana incluye corrección + tests para los items correspondientes.